Design the PUT Endpoint for Updating an Existing Product

Understand how to design and implement a PUT endpoint for updating an existing product in a REST API using Python and Flask. Learn the key features of the PUT method, request and response structure, appropriate status codes, and best practices for handling updates in RESTful services.

To update an existing entry in the collection, we use the PUT method.

Updating an existing product

According to the specifications, “The PUT method requests that the enclosed entity be stored under the supplied Request-URI. If the Request-URI refers to an already existing resource, the enclosed entity SHOULD be considered as a modified version of the one residing on the origin server. If the Request-URI does not point to an existing resource, and that URI is capable of being defined as a new resource by the requesting user agent, the origin server can create the resource with that URI. If a new resource is created, the origin server MUST inform the user agent via the 201 (Created) response.”

So, PUT can be used for creating a new product as well as updating an existing product. However, creating a new product using PUT is not generally recommended or practised.

Ask

Based on the above definition, the following are the main features of the PUT method:

- The Request-URI refers to an existing item.
- The entity provided within the body has to be used to update the existing item referred to via the resource URI.
- PUT is idempotent, i.e., multiple PUT requests with the same data will result in the same effect on the server as a single PUT request.
- If the updating of an existing entity is encapsulated in a long process, then the initial response will be `202 ACCEPTED`, with a URI to know the current status. The status URI should send either `200 OK` if the operation is successful, or send the corresponding error otherwise.
- The request and the response can't be cached.

Request/response structure

Let's look at the request and response structure of the header and body of the PUT method.

Request header

The headerS of a PUT request are similar to POST, and these are:

- **Method:** This is the HTTP method to be executed by the server. It'll be PUT in this case.
- **URL/URI:** This is the identifying URI of the resource. If the URI exists, then the value of this header specifies that the entry identified by the URI needs to be updated. If the URI doesn't exist, the general practice is to send back `404 NOT FOUND`.
- **Version:** This is the version of the HTTP, i.e., 1.0, 1.1, and so on.
- **Content-Type:** – This is the MIME type/format of the content.
- **Content-Length:** – This is the length/size of the enclosed entity or content.
- **Accept:** - This is the MIME type/format of the body of the response t client will accept. This becomes important if the service returns the updated entity as a part of the response.

Ask

Request body

The body of the request contains the data to be changed. The format is specified in the header. If there's a mismatch between the format of the data in the body and the format specified in the header, the server can reject the request.

Similarly, the PUT response sent by the server will also have a header and a body. The PUT response tells the client whether the request has been successful or not, the entity that was updated, or the reason for the failure of the request.

The server has to keep the following in mind while sending the PUT response:

- If an entity has been updated as a result of the processing of the request, the server should return the entity as part of the response body.
- The response is not cacheable.

Response header

Unlike POST, in a PUT response, the headers `Cache-Control` and `Location` are not present. The other headers are the same, which are:

- **Content-Type** This is the format or MIME type of content in the body. This should be the same as that of the `Accept` header of the request.
- **Content-Length:** This is the size of the content within the body.
- **Status:** This is the status of the request. It tells whether the request has been successfully processed or not. In the case of PUT, the initial response status should be `202 ACCEPTED`. It means that the server has accepted the request for processing. Then, if the URI refers to an existing entry and the update is successful, the status should be `200 OK`. If the URI doesn't refer to an existing entry, then the status should be `404 NOT FOUND`. The sequence of `202` succeeded by `200` is decided by the server. The server can decide to send `200` without initially sending `202` if the processing won't take a lo time.

Ask

> **Note:** Among the above, `Status` is the most important since it indicates the successful processing of the request.

Response body

The body part is optional for PUT. The server doesn't need to return the updated entity in the body.

> **Note:** In practice, most servers send back the body to avoid a secondary call to retrieve the updated data. If the body is present, its MIME type should be the same as the MIME type specified by the `Accept` header of the request.

Ask

Request/response structure for PUT API for updating an existing product

Products API

Keeping the above points in mind, the request header for updating an existing product will be as follows:

index.js Javascript (babel-node)

```
PUT /api/products/3 HTTP/1.1
Host: localhost:5000
User-Agent: curl/7.71.1
Content-Type: application/json
Content-Length: 20
Accept: application/json
```

The `User-Agent` header will be optional.

The request body sent by the client will be:

index.js Javascript (babel-node)

```
{
        "name": "rest-container",
        "description": "container for rest",
        "version": "3.0"
}
```

Ask

> **Note:** Please note that version has been updated from 2.2 to 3.0.

The response header provided by the server will be:

index.js Javascript (babel-node)

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 92
```

The response body sent by the server will be:

index.js Javascript (babel-node)

```
{
     "product":{
                 "id":"3",
                 "name": "rest-container",
                 "description": "container for rest",
                 "version": "3.0"
                }
}
```

Ask

> **Note:** The data given above is just for the sake of example.

Mark As Completed

Ask